



INDEPENDENT SECURITY REVIEW

September 29, 2026

LI.FI

LiFilntentEscrowFacetV2 v2.0.0

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1	About Researcher	1
2	Disclaimer	1
3	Scope	1
4	Risk classification	1
4.1	Impact	1
4.2	Likelihood	2
4.3	Action required for severity levels	2
5	Executive Summary	2
6	Findings	3
6.1	Low Risk	3
6.1.1	The LI.FI event does not identify the full Stellar recipient	3
6.1.2	Exclusive contexts accept arbitrary trailing data	3
6.2	Informational	3
6.2.1	The documented output-context format no longer matches the code	3
6.2.2	Stellar muxed-recipient handling lacks end-to-end coverage	3
6.2.3	Swap refunds can sweep pre-existing Diamond balances	4
6.2.4	Native TRX routes use an incompatible input settler	4

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Valinor, Kuru, MNX, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols. He has uncovered live bugs in Ripple, Uniswap, Morpho, Phantom, Cosmos and received bug-bounties from numerous companies.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

LiFiIntentEscrowFacetV2	src/Facets/LiFiIntentEscrowFacetV2.sol (v2.0.0)
-------------------------	---

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like grieving attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 4 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is a differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **6 issues** were found.

Project Summary

Project Name	LiFiIntentEscrowFacetV2 v2.0.0
Project Type	Solidity Smart Contracts
Repository	lifinance/contracts
Commit	331df8131
Audit Timeline	September 28, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Gas Optimizations	0
Informational	4
Total Issues	6

6 Findings

6.1 Low Risk

6.1.1 The LI.FI event does not identify the full Stellar recipient

CONTEXT: [LiFiIntentEscrowFacetV2.sol#L212-L223](#)

DESCRIPTION: Two muxed recipients sharing the same Stellar base account produce the same LI.FI receiver event. Indexers relying only on that event cannot identify the intended sub-account.

RECOMMENDATION: Emit the muxed ID in a dedicated event, or clearly require Stellar integrations to reconstruct the recipient from the origin settler's `Open` event.

LI.FI: Documented in [fcd05b9d0](#) that delivery data lives in the opaque destination context and must be read from the origin settler's `Open` event.

RESEARCHER: Verified documentation.

6.1.2 Exclusive contexts accept arbitrary trailing data

CONTEXT: [LiFiIntentEscrowFacetV2.sol#L302-L315](#)

DESCRIPTION: The intended Stellar context is 45 bytes, but the facet accepts every length above 37. A malformed suffix or a context paired with the wrong settler can open successfully and later become unfillable, leaving the input escrowed until refund.

RECOMMENDATION: Accept only 37 or 45 bytes if the muxed ID is the sole extension. If more formats are expected, validate the format against the selected settler or add an explicit version.

LI.FI: Acknowledged: Intentional. Trailing bytes are forwarded unchanged so that new settler context formats don't need a facet change. Pairing a context with the right settler is the integrator's job; a mismatch ends in a refund.

RESEARCHER: Acknowledged.

6.2 Informational

6.2.1 The documented output-context format no longer matches the code

CONTEXT: [LiFiIntentEscrowFacetV2.sol#L302-L315](#), [LiFiIntentEscrowFacetV2.md#L72-L81](#).

DESCRIPTION: The facet documentation still says an exclusive context must be exactly 37 bytes and that any other length reverts. Integrators following it cannot construct the new Stellar format correctly.

RECOMMENDATION: Document the supported Stellar layout, total length, byte order, and how the full muxed recipient is recovered from emitted order data.

LI.FI: Fixed in [0866af3c8](#).

RESEARCHER: Verified fix.

6.2.2 Stellar muxed-recipient handling lacks end-to-end coverage

CONTEXT: [LiFiIntentEscrowFacetV2.t.sol#L1126-L1141](#)

DESCRIPTION: The tests do not prove that the muxed ID survives both entrypoints with the expected byte order or that the Stellar settler can decode and fill the resulting order. The public intent encoder also still produces only the original 37-byte context.

RECOMMENDATION: Add 45-byte tests for both entrypoints, relative and absolute deadlines, and zero/max muxed IDs. Include a fill test against the actual Stellar settler and update the encoder.

LI.FI: Acknowledged: Stellar muxed-recipient decoding and fills will be tested on the Stellar side.

RESEARCHER: Acknowledged.

6.2.3 Swap refunds can sweep pre-existing Diamond balances

CONTEXT: SwapperV2.sol#L336-L353

DESCRIPTION: For non-final input assets, the helper refunds the entire remaining balance instead of only the amount introduced by the current call. A caller can include an asset already held by the Diamond and route that balance to an address they control.

RECOMMENDATION: Snapshot every input asset balance before the swaps and refund only the positive balance increase created by the current call. Since this is a known-limitation, this issue could also be acknowledged.

LI.FI: Acknowledged. Known LI.FI Diamond issue. The LI.FI Diamond should never hold assets; this facet does not change that assumption.

RESEARCHER: Acknowledged.

6.2.4 Native TRX routes use an incompatible input settler

CONTEXT: LiFiIntentEscrowFacetV2.sol#L276-L291

DESCRIPTION: The configured Tron input settler does not support a payable `open`. Native TRX orders will therefore revert even though the facet advertises native input support.

RECOMMENDATION: Deploy and configure a native-aware Tron settler before enabling this facet on Tron. Until then, reject native TRX explicitly and keep the limitation documented.

LI.FI: Acknowledged: Tron is deployed from <https://github.com/lifinance/contracts-tron>, which is still on v1.0.0 (ERC20 inputs only). Native TRX support will be settled when v2.0.0 is ported there.

RESEARCHER: Acknowledged.